

A-Z Linear Algebra & Calculus for AI, Data Science and Machine Learning

Instructor:

Rashedul Alam Shakil

Founder of Study Mart

Founder of aiQuest Intelligence

Master in Data Science at FAU Erlangen

Automation Programmer at Siemens Energy, Germany

High-Dimensional Statistics for Machine Learning

[Watch Tutorial](#)

What is High-Dimensional Data?

Machine Learning

High-Dimensional Statistics is a field of statistics that deals with data where the **number of variables** (features or parameters) is **very large**, often comparable to or **larger than the number of observations**.

Mathematically, Features >> Samples

In classical statistics, you usually have more observations than variables, for example, 1000 patients but only 5 measurements per patient. High-dimensional statistics flips this: you might have 1000 measurements for only 50 patients. This happens a lot in modern applications like:

- Genomics (thousands of genes, few samples)
- Finance (many assets, short periods)
- Machine Learning (massive feature spaces from text, images, etc.)

What is High-Dimensional Data?

Machine Learning

When the number of features (variables) is **much greater than the number of samples**, a high-dimensional setting, several traditional machine learning models tend to **underperform or fail** due to overfitting, computational issues, or lack of generalizability. Models that typically **do not work well** in such scenarios include:

- **1. Ordinary Least Squares (OLS) Linear Regression**
 - **Problem:** Cannot compute a unique solution when features > samples.
 - **Reason:** The matrix $X^T X$ is not invertible (singular) in high dimensions.
- **2. Standard Logistic Regression**
 - Suffers from overfitting when too many features are present without regularization.
- **3. k-Nearest Neighbors (k-NN)**
 - **Problem:** Suffers from the **curse of dimensionality**, distances between points become meaningless in high dimensions.
- **4. Naive Bayes (in some forms)**
 - May be overwhelmed by noise in irrelevant features unless strong independence assumptions hold.
- **5. Unregularized Decision Trees**
 - Tend to overfit badly due to the many possible feature splits.

What is High-Dimensional Data?

Machine Learning

System of Equations:

$$x + y + z + w = 5$$

$$2x + 3y + z + 2w = 10$$

$$-x + y + 2z + w = 3$$

Augmented Matrix Form:

$$\left[\begin{array}{cccc|c} 1 & 1 & 1 & 1 & 5 \\ 2 & 3 & 1 & 2 & 10 \\ -1 & 1 & 2 & 1 & 3 \end{array} \right]$$

High-Dimensional Statistics is a field of statistics that deals with data where the **number of variables** (features or parameters) is **very large**, often comparable to or **larger than the number of observations**.

text	
0	Eddard Stark is a king in the north.
1	A king but one king : kings are everywhere.
2	Hodor was different : he was not a king .
3	But the North could not change without him.

sparse

	king	was	the	not	But	him	one	north	kings	is	in	he	Eddard	everywhere	different	could	change	but	are	Stark	North	Hodor	without
0	1	0	1	0	0	0	0	1	0	1	1	0	1	0	0	0	0	0	0	1	0	0	0
1	2	0	0	0	0	0	1	0	1	0	0	0	0	1	0	0	0	1	1	0	0	0	0
2	1	2	0	1	0	0	0	0	0	0	0	1	0	0	1	0	0	0	0	0	0	1	0
3	0	0	1	1	1	1	0	0	0	0	0	0	0	0	0	1	1	0	0	0	1	0	1

High-Dimensional Statistics is a field of statistics that deals with data where the **number of variables** (features or parameters) is **very large**, often comparable to or **larger than the number of observations**.

Important concepts:

- **Regularization:** Adding penalties (like Lasso, Ridge) to control model complexity.
- **Dimensionality reduction:** Methods like PCA (Principal Component Analysis) reduce the number of variables.
- **Sparse modelling:** If only a few features matter and trying to find them.
- **Concentration of measure:** In high dimensions, random variables behave differently (e.g., distances between points become almost the same).

What is Regularization?

Lasso Vs. Ridge Vs. Elastic Net

[Watch Tutorial](#)

What is Regularization?

High-Dim Machine Learning

Regularization is a technique used in machine learning and statistics to **prevent overfitting**, which occurs when a model learns the training data, including its noise and outliers, too well and performs poorly on unseen data.

Why Is Regularization Needed?

When a model is too complex (a very deep neural network or a high-degree polynomial), it might:

- Fit the training data very well (low training error),
- But generalize poorly to new, unseen data (high test error).

What is Regularization?

High-Dim Machine Learning

Regularization is a technique used in machine learning and statistics to **prevent overfitting**, which occurs when a model learns the training data, including its noise and outliers, too well and performs poorly on unseen data.

How Regularization Works

Regularization adds a **penalty term** to the model's loss function that discourages overly complex models. It effectively **shrinks the model weights**, pushing them toward zero (but not exactly zero unless using specific types like L1).

1. L1 Regularization (Lasso)

It adds a penalty equal to the absolute value of the coefficients. This encourages sparsity (some weights become **exactly zero**). **Coordinate descent** is effective because it updates one weight at a time and can handle this non-smooth behaviour.

$$\text{Loss} = \text{Original Loss} + \lambda \sum |w_i|$$
$$\text{Loss} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^m |w_j|$$

- $\lambda \sum_{j=1}^m |w_j|$: **L1 penalty**, where:
 - w_j : Model weights
 - λ : Regularization strength
 - m : Number of weights/features

2. L2 Regularization (Ridge)

Adds a penalty equal to the **square** of the coefficients. Encourages smaller weights, but not necessarily zero.

$$\text{Loss} = \text{Original Loss} + \lambda \sum w_i^2$$

$$\text{Loss} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^m w_j^2$$

The L2 penalty is smooth and differentiable, so you can solve Ridge Regression directly with linear algebra (**Closed-form solution**) if the data isn't too large.

$$w = (X^T X + \lambda I)^{-1} X^T y$$

3. Elastic Net:

Elastic Net uses a linear regression model but optimizes it using a combination of L1 and L2 penalties. The algorithm most used to solve Elastic Net is a variation of **coordinate descent**. The cost function for

Elastic Net regularization, which combines both **L1** and **L2** regularization with **MSE**:

$$\text{Loss} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda_1 \sum_{j=1}^m |w_j| + \lambda_2 \sum_{j=1}^m w_j^2$$

Components:

- $\frac{1}{n} \sum (y_i - \hat{y}_i)^2$: Standard **MSE loss**
- $\lambda_1 \sum |w_j|$: **L1 penalty** (promotes sparsity)
- $\lambda_2 \sum w_j^2$: **L2 penalty** (promotes small weights)

3. Elastic Net:

cost function for **Elastic Net regularization**, which combines both **L1** and **L2** regularization with **MSE**:

Why Use Elastic Net?

- Useful when:
 - You suspect some features are irrelevant (L1 helps by eliminating them).
 - You also want to maintain stability and avoid over-reliance on a small set of features (L2 helps with that).
 - Especially effective in **high-dimensional datasets** where the number of features is much larger than samples

Regularization	Main Algorithm Used	Important Notes
01. L1 (Lasso)	Coordinate Descent, LARS	Good for feature selection; handles sparsity well
02. L2 (Ridge)	Closed-form, Gradient Descent	Smooth, shrinks weights but keeps all features
03. Elastic Net	Coordinate Descent	Combines strengths of both L1 and L2

In Deep Learning

- **Dropout** is a type of regularization where random neurons are ignored during training.
- **Weight decay** is a form of L2 regularization.

What is Thresholding?

[Watch Tutorial](#)

Thresholding is a technique **used to simplify or sparsify data by setting small values to zero** and keeping or shrinking larger values. It is widely used in statistics, signal processing, and machine learning to remove noise or weak relationships.

Hard and soft thresholding are **alternative** approaches for inducing sparsity, similar in spirit to Lasso and Ridge, though **not always used** in the same optimization frameworks.

Types of Thresholding:

- Hard Thresholding
- Soft Thresholding

$$T_{\lambda}(u) = u \cdot \mathbf{1}_{\{|u| \geq \lambda\}}$$

- This means:
 - Keep u **only** if $|u| \geq \lambda$
 - Otherwise, set it to 0
- Discrete cutoff: **either full value or zero**
- **Used for aggressive sparsity**

Hard and Soft Thresholding

Hard Thresholding

COV Matrix: $\Sigma = \frac{1}{8} \begin{pmatrix} 4 & -2 & -2 & -2 & -2 \\ -2 & 5 & 1 & 1 & 1 \\ -2 & 1 & 5 & 1 & 1 \\ -2 & 1 & 1 & 5 & 1 \\ -2 & 1 & 1 & 1 & 5 \end{pmatrix} = \begin{pmatrix} 0.5 & -0.25 & -0.25 & -0.25 & -0.25 \\ -0.25 & 0.625 & 0.125 & 0.125 & 0.125 \\ -0.25 & 0.125 & 0.625 & 0.125 & 0.125 \\ -0.25 & 0.125 & 0.125 & 0.625 & 0.125 \\ -0.25 & 0.125 & 0.125 & 0.125 & 0.625 \end{pmatrix}$

Threshold (λ) = 0.125

Keep values where $|u| \geq \lambda = 0.125$, otherwise set to 0.

$$T_{\lambda}(\Sigma) = \begin{pmatrix} 0.5 & -0.25 & -0.25 & -0.25 & -0.25 \\ -0.25 & 0.625 & 0.125 & 0.125 & 0.125 \\ -0.25 & 0.125 & 0.625 & 0.125 & 0.125 \\ -0.25 & 0.125 & 0.125 & 0.625 & 0.125 \\ -0.25 & 0.125 & 0.125 & 0.125 & 0.625 \end{pmatrix}$$

$$S_{\lambda}(u) = \text{sign}(u) \cdot (|u| - \lambda) \cdot \mathbf{1}_{\{|u| \geq \lambda\}}$$

The notation $\mathbf{1}_{\{|u| \geq \lambda\}}$ is called an **indicator function**.

It returns 1 if the condition inside is true, and 0 if the condition is false.

- This means:
 - If $|u| \geq \lambda$, shrink it by λ in magnitude, keeping the same sign
 - If $|u| < \lambda$, set it to 0
- Smoother than hard thresholding
- This is the **core step** in **Lasso's coordinate descent algorithm**

COV Matrix: $\Sigma = \frac{1}{8} \begin{pmatrix} 4 & -2 & -2 & -2 & -2 \\ -2 & 5 & 1 & 1 & 1 \\ -2 & 1 & 5 & 1 & 1 \\ -2 & 1 & 1 & 5 & 1 \\ -2 & 1 & 1 & 1 & 5 \end{pmatrix} = \begin{pmatrix} 0.5 & -0.25 & -0.25 & -0.25 & -0.25 \\ -0.25 & 0.625 & 0.125 & 0.125 & 0.125 \\ -0.25 & 0.125 & 0.625 & 0.125 & 0.125 \\ -0.25 & 0.125 & 0.125 & 0.625 & 0.125 \\ -0.25 & 0.125 & 0.125 & 0.125 & 0.625 \end{pmatrix}$

Threshold (λ) = 0.125

$$S_{\lambda}(\Sigma) = \begin{pmatrix} 0.375 & -0.125 & -0.125 & -0.125 & -0.125 \\ -0.125 & 0.5 & 0 & 0 & 0 \\ -0.125 & 0 & 0.5 & 0 & 0 \\ -0.125 & 0 & 0 & 0.5 & 0 \\ -0.125 & 0 & 0 & 0 & 0.5 \end{pmatrix}$$

Apply to each entry:

- $0.5 \rightarrow 0.5 - 0.125 = 0.375$
- $-0.25 \rightarrow -0.25 + 0.125 = -0.125$
- $0.125 \rightarrow 0.125 - 0.125 = 0$

The sign function simply **tells you the direction** (positive or negative) of a number:

$$\text{sign}(u) = \begin{cases} +1, & u > 0 \\ 0, & u = 0 \\ -1, & u < 0 \end{cases}$$

1. $u = 0.5$

- $\text{sign}(0.5) = +1$
- So: $S_\lambda(0.5) = 1 \cdot (0.5 - 0.125) = 0.375$

The sign function simply tells you the direction (positive or negative) of a number:

$$\text{sign}(u) = \begin{cases} +1, & u > 0 \\ 0, & u = 0 \\ -1, & u < 0 \end{cases}$$

1. $u = 0.5$

- $\text{sign}(0.5) = +1$
- So: $S_\lambda(0.5) = 1 \cdot (0.5 - 0.125) = 0.375$

2. $u = -0.25$

- $\text{sign}(-0.25) = -1$
- So: $S_\lambda(-0.25) = -1 \cdot (0.25 - 0.125) = -1 \cdot 0.125 = -0.125$

Hard and Soft Thresholding

How sign(u) works?

The sign function simply **tells you the direction** (positive or negative) of a number:

$$\text{sign}(u) = \begin{cases} +1, & u > 0 \\ 0, & u = 0 \\ -1, & u < 0 \end{cases}$$

1. $u = 0.5$

- $\text{sign}(0.5) = +1$
- So: $S_\lambda(0.5) = 1 \cdot (0.5 - 0.125) = 0.375$

2. $u = -0.25$

- $\text{sign}(-0.25) = -1$
- So: $S_\lambda(-0.25) = -1 \cdot (0.25 - 0.125) = -1 \cdot 0.125 = -0.125$

3. $u = 0.125$

- $\text{sign}(0.125) = 1$
- $|0.125| - 0.125 = 0$
- So: $S_\lambda(0.125) = 1 \cdot 0 = 0$

Hard and Soft Thresholding

What if we won't use the $\text{sign}(u)$?

Suppose we used this instead:

$$S_{\lambda}(u) = |u| - \lambda$$

Now apply it to $u = -0.25$:

- $|u| = 0.25$
- $|u| - \lambda = 0.125$

You'd get **+0.125**, which is **incorrect**; you just turned a **negative** value into a **positive** one!

The **sign function is essential** in soft thresholding because it ensures the **output keeps the same sign** (positive or negative) as the original input. Without it, all values would become positive after shrinking, which would **destroy the original structure or relationships** in the data.